

Documento de Contexto Técnico: Simulación de Onda de Péndulos

Corpus de Conocimiento para IA

12 de diciembre de 2025

Resumen

Este documento constituye un Corpus de Conocimiento.º Contexto Maestro". Su propósito es doble: servir para el *fine-tuning* de modelos especializados en simulaciones físicas y proporcionar un contexto avanzado para *Prompt Engineering* (Few-Shot / Chain-of-Thought). El objetivo es permitir que un modelo de IA deduzca lógicamente el código de simulación basándose en principios exactos.

Índice

1. Marco Teórico y Físico	1
1.1. El Péndulo Simple y la Aproximación de Ángulos Pequeños	1
1.2. Transformación de Coordenadas Polares a Cartesianas	2
1.3. El Efecto "Pendulum Wave" (Onda de Péndulo)	2
2. Marco Computacional y Librerías (Python)	2
2.1. NumPy (<code>import numpy as np</code>)	2
2.2. Matplotlib Pyplot (<code>import matplotlib.pyplot as plt</code>)	3
2.3. Matplotlib Animation	3
2.4. Integración en Google Colab	3
3. Instrucción de Síntesis (El Prompt Final)	4

1. Marco Teórico y Físico

1.1. El Péndulo Simple y la Aproximación de Ángulos Pequeños

El sistema base para esta simulación es el péndulo simple idealizado: una masa puntual suspendida de una cuerda sin masa e inextensible de longitud L , oscilando bajo la influencia de la gravedad g .

La ecuación diferencial que rige el movimiento es no lineal:

$$\frac{d^2\theta}{dt^2} + \frac{g}{L} \sin(\theta) = 0 \quad (1)$$

Sin embargo, el código a replicar utiliza la **Aproximación de Ángulos Pequeños**. Para ángulos iniciales pequeños ($\theta < 20^\circ$ o $\approx 0,35$ rad), se asume que $\sin(\theta) \approx \theta$. Esto linealiza la

ecuación diferencial convirtiéndola en un Oscilador Armónico Simple:

$$\frac{d^2\theta}{dt^2} + \frac{g}{L}\theta = 0 \quad (2)$$

La solución analítica exacta para esta aproximación, utilizada en el script, es:

$$\theta(t) = \theta_0 \cos(\omega t) \quad (3)$$

Donde:

- $\theta(t)$ es el ángulo en el tiempo t .
- θ_0 es la amplitud inicial.
- ω es la frecuencia angular natural, definida como:

$$\omega = \sqrt{\frac{g}{L}} \quad (4)$$

1.2. Transformación de Coordenadas Polares a Cartesianas

Para graficar el péndulo en una pantalla (espacio 2D), se deben convertir las coordenadas angulares (θ, L) a cartesianas (x, y) :

$$x = L \cdot \sin(\theta) \quad (5)$$

$$y = -L \cdot \cos(\theta) \quad (6)$$

Nota: El signo negativo en y se debe a que el punto de reposo del péndulo está "bajo" del origen en la gráfica.

1.3. El Efecto “Pendulum Wave” (Onda de Péndulo)

La simulación no es de un solo péndulo, sino de un conjunto de N péndulos. La belleza visual (patrones de serpiente, caos y reordenamiento) surge porque el periodo de oscilación depende de la raíz cuadrada de la longitud:

$$T = 2\pi\sqrt{\frac{L}{g}} \quad (7)$$

Al definir un arreglo de longitudes L linealmente espaciadas (usando `np.linspace`), cada péndulo tiene una frecuencia ligeramente diferente a su vecino. Esto provoca un desfase progresivo que crea patrones visuales de ondas viajeras.

2. Marco Computacional y Librerías (Python)

Para replicar el código, la IA debe comprender la función específica de cada librería en el contexto de Google Colab.

2.1. NumPy (`import numpy as np`)

Es el motor de cálculo vectorial. En lugar de usar bucles `for` lentos para calcular la posición de cada péndulo en cada milisegundo, el código aprovecha la vectorización:

- `np.linspace(start, stop, num)`: Genera el vector de longitudes de las cuerdas.
- `np.arange(start, stop, step)`: Crea el vector de tiempo discreto.
- `np.sqrt()` y `np.cos()`: Funciones universales (ufuncs) que operan sobre arrays completos instantáneamente.

2.2. Matplotlib Pyplot (`import matplotlib.pyplot as plt`)

Encargado de la estructura estática de la visualización:

- `fig, ax = plt.subplots()`: Crea el lienzo y los ejes.
- `ax.plot()`: Inicialmente se usa para crear objetos vacíos (líneas y puntos) que luego serán animados. Es crucial guardar las referencias a estos objetos (`lines`, `masses`) para modificarlos después.

2.3. Matplotlib Animation

El núcleo de la dinámica (`from matplotlib.animation import FuncAnimation`). Funciona bajo el concepto de "Blitting"(redibujado optimizado):

- **Función `update(frame)`**: Es el "game loop". Recibe el número de cuadro actual, calcula las nuevas coordenadas (x, y) usando la física descrita arriba y actualiza los datos de los objetos gráficos usando `.set_data()`.
- **Eficiencia**: Al usar `lines[i].set_data()`, se evita borrar y crear la gráfica completa en cada cuadro, lo cual es vital para una animación fluida.

2.4. Integración en Google Colab

Google Colab es un entorno web estático por defecto. Se usa `from IPython.display import HTML`.

- `ani.to_jshtml()`: Convierte la animación de Python en código JavaScript/HTML interactivo.
- `HTML(...)`: Renderiza ese bloque dentro de la celda del notebook, permitiendo controles de "Play/Pause".

3. Instrucción de Síntesis (El Prompt Final)

Este es el bloque diseñado para ser entregado a la IA. Combina la teoría y la técnica para forzar la generación exacta del código.

INSTRUCCIÓN DEL SISTEMA

ROL: Eres un Asistente Experto en Desarrollo de Software Científico y Física Computacional.

OBJETIVO: Generar un script de Python ejecutable en Google Colab que simule el fenómeno de "Pendulum Wave" (Onda de Péndulo) utilizando múltiples osciladores armónicos simples.

ESPECIFICACIONES DEL CÓDIGO (Constraints):

- **Entorno:** El código debe ser compatible con Jupyter Notebooks / Google Colab.
- **Lógica Matemática:**
 - No uses integración numérica (como Runge-Kutta). Usa la solución analítica para pequeñas oscilaciones: $\theta(t) = \theta_0 \cos(\sqrt{g/L} \cdot t)$.
 - Debes simular entre 1 y 15 péndulos. Define `num_pendulos = 10` como variable fija, asegurando con `max(1, min(15, ...))` que se mantenga en rango.
 - Genera las longitudes L en un rango de 1.0 a 2.0.
 - Parámetros: $g = 9,81$, $t_{max} = 20$, $dt = 0,02$, $\theta_0 = 0,35$ rad.
- **Pre-cálculo:** Calcula todas las posiciones angulares `thetas` antes de iniciar la animación para mejorar el rendimiento.
- **Visualización (Matplotlib):**
 - Configura el canvas con `xlim (-2.5, 2.5)` y `ylim (-2.5, 0.5)`.
 - Dibuja un soporte fijo en `(0,0)`.
 - Genera colores distintos para cada péndulo usando `plt.cm.viridis`.
 - Crea objetos `line` (para la cuerda) y `mass` (para la esfera) vacíos inicialmente.
- **Animación (FuncAnimation):**
 - Crea una función `update(frame)` que iterativamente actualice las coordenadas (x, y) de cada péndulo:
$$x = L \sin(\theta)$$
$$y = -L \cos(\theta)$$
 - Usa `set_data` para actualizar las gráficas.
 - La función debe retornar una lista combinada de líneas y masas.
- **Renderizado Web:**
 - Cierra la figura estática con `plt.close()`.
 - Muestra la animación final convirtiéndola a JavaScript HTML (`ani.to_jshtml()`) y renderizándola con `IPython.display.HTML`.

SALIDA ESPERADA: Proporciona únicamente el bloque de código Python completo, comentado y listo para copiar y pegar.

¿Cómo usar esto?

Si copias y pegas todo el contenido anterior (desde "Marco Teórico" hasta el final del "Prompt") en una IA avanzada, le estarás dando una clase magistral sobre tu código. La IA no solo escribirá el código, sino que entenderá por qué funciona, permitiéndote luego pedirle modificaciones complejas como: "Ahora añade rozamiento con el aire." o "Cambia la gravedad a la de la Luna", y la IA sabrá exactamente qué fórmula matemática ajustar.